

MACHINE LEARNING FUNDAMENTALS SERIES

ArgMax & SoftMax: Foundations, Derivatives & Optimization

Multi-Class Classification, Derivative Derivation via Quotient Rule, and Training Objectives

1. Introduction and Problem Formulation

In multi-class classification neural networks (e.g., predicting Iris species: *Setosa*, *Versicolor*, or *Virginica*), the raw output layer produces unbounded real numbers $z_1, z_2, \dots, z_K$. For example, raw scores might be:

$$z_1 = 1.43 \quad (\text{Setosa}), \quad z_2 = -0.40 \quad (\text{Versicolor}), \quad z_3 = 0.23 \quad (\text{Virginica})$$

Because raw scores can be negative or greater than 1, they are difficult to interpret directly. To convert raw outputs into a clear classification decision or normalized probability distribution, the network routes these scores to either an **ArgMax** or a **SoftMax** layer.

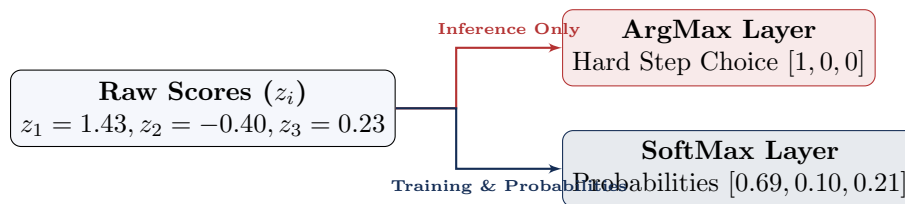


Figure 1: Routing raw neural network scores through ArgMax vs. SoftMax layers.

2. ArgMax vs. SoftMax: Mathematical Definitions

2.1. ArgMax (Arguments of the Maximum)

The **ArgMax** function assigns a hard 1 to the class with the highest raw score and 0 to all other classes:

$$\text{ArgMax}(z_i) = \begin{cases} 1 & \text{if } z_i = \max(z_1, \dots, z_K) \\ 0 & \text{otherwise} \end{cases} \implies \text{Output}_{\text{ArgMax}} = [\text{Setosa: } 1, \text{ Versicolor: } 0, \text{ Virginica: } 0]$$

Why ArgMax CANNOT Be Used for Backpropagation

Although ArgMax provides a crisp prediction, **it cannot be used to optimize weights via Gradient Descent.**

- Zero Slope:** On either side of the decision boundary, ArgMax is constant (0 or 1). Its derivative is strictly **zero**: $\frac{\partial \text{ArgMax}(z_i)}{\partial z_i} = 0$.
- Chain Rule Collapse:** In Backpropagation, computing the loss gradient relative to any weight w requires multiplying by the ArgMax derivative:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \text{ArgMax}} \cdot \underbrace{\frac{\partial \text{ArgMax}}{\partial z_i}}_0 \cdot \frac{\partial z_i}{\partial w} = 0$$

- Paralyzed Gradient Descent:** Step size equals $\Delta w = -\eta \cdot \frac{\partial \mathcal{L}}{\partial w} = -\eta \cdot 0 = 0$. The optimizer cannot update parameters.

2.2. SoftMax (Soft Maximum Probability Distribution)

To enable gradient optimization, **SoftMax** replaces hard step choices with a smooth, differentiable probability distribution:

$$S_i = \text{SoftMax}(z_i) = \frac{e^{z_i}}{\sum_{k=1}^K e^{z_k}}$$

Key Properties: (1) **Preserves Rank** (highest $z_i \implies$ highest S_i); (2) **Bounded** ($0 < S_i < 1$); (3) **Sums to 1** ($\sum S_k = 1.0$).

Numerical Example for Iris Scores ($z_1 = 1.43, z_2 = -0.40, z_3 = 0.23$):

$$\sum e^{z_k} = e^{1.43} + e^{-0.40} + e^{0.23} \approx 4.179 + 0.670 + 1.259 = 6.108$$
$$S_1 = \frac{4.179}{6.108} = \mathbf{0.69}, \quad S_2 = \frac{0.670}{6.108} = \mathbf{0.10}, \quad S_3 = \frac{1.259}{6.108} = \mathbf{0.21}$$

3. Step-by-Step SoftMax Derivative Derivation

Because the denominator $\sum e^{z_k}$ contains all raw scores, changing z_j affects the SoftMax probability S_i for all classes. We derive both cases using the **Quotient Rule**: $\frac{d}{dz} \left(\frac{u}{v} \right) = \frac{u'v - uv'}{v^2}$.

3.1. Case 1: Derivative wrt Same Class Score ($i = j$)

Let $S_i = \frac{e^{z_i}}{\sum e^{z_k}}$. Differentiating wrt z_i ($u = e^{z_i} \implies u' = e^{z_i}$; $v = \sum e^{z_k} \implies v' = e^{z_i}$):

$$\frac{\partial S_i}{\partial z_i} = \frac{e^{z_i} (\sum e^{z_k}) - e^{z_i} e^{z_i}}{(\sum e^{z_k})^2} = \frac{e^{z_i}}{\sum e^{z_k}} - \left(\frac{e^{z_i}}{\sum e^{z_k}} \right)^2 = \mathbf{S_i(1 - S_i)}$$

Numerical Check for Setosa ($S_1 = 0.69$): $\frac{\partial S_1}{\partial z_1} = 0.69 \times (1 - 0.69) = 0.69 \times 0.31 = \mathbf{0.21}$.

3.2. Case 2: Derivative wrt Different Class Score ($i \neq j$)

Differentiating S_i wrt a different score z_j ($u = e^{z_i} \implies u' = 0$; $v = \sum e^{z_k} \implies v' = e^{z_j}$):

$$\frac{\partial S_i}{\partial z_j} = \frac{0 \cdot (\sum e^{z_k}) - e^{z_i} e^{z_j}}{(\sum e^{z_k})^2} = - \left(\frac{e^{z_i}}{\sum e^{z_k}} \right) \left(\frac{e^{z_j}}{\sum e^{z_k}} \right) = \mathbf{-S_i S_j}$$

Numerical Check for Setosa wrt Versicolor ($S_2 = 0.10$) **and** **Virginica** ($S_3 = 0.21$):

$$\frac{\partial S_1}{\partial z_2} = -(0.69 \times 0.10) = \mathbf{-0.07}, \quad \frac{\partial S_1}{\partial z_3} = -(0.69 \times 0.21) = \mathbf{-0.15}$$

4. Optimization Objective: Cross-Entropy Loss vs. SSR

In earlier networks predicting continuous outputs, model parameters were optimized using Sum of Squared Residuals (SSR).

Shift to Cross-Entropy Loss for Classification

When a neural network utilizes a SoftMax output layer, the predicted outputs represent a probability distribution ($0 < S_i < 1$). In this classification paradigm, parameter updating via Gradient Descent uses **Cross-Entropy Loss** rather than Sum of Squared Residuals (SSR). Cross-Entropy penalizes confident incorrect predictions far more effectively and combines with SoftMax to yield clean, elegant gradient updates during Backpropagation.

5. Summary Comparison: ArgMax vs. SoftMax

Property / Aspect	ArgMax Layer	SoftMax Layer
Output Type	Discrete Binary Vector (0 or 1)	Continuous Probability Vector ($0 < S_i < 1$)
Sum of Outputs	Equals 1 (Exactly one 1, rest 0)	Equals 1.0 ($\sum S_i = 1$)
Differentiability	Non-differentiable (Slope = 0)	Smoothly Differentiable ($S_i(1 - S_i)$ or $-S_i S_j$)
Primary Role	Final Inference / Classification	Network Training / Optimization
Loss Function	N/A (Cannot be trained)	Cross-Entropy Loss